CR TACHE2 :

Voici le contenu de geom2d.h

```
#ifndef _GEOM2D_H_
#define _GEOM2D_H_
#include « types_macros.h"

typedef struct Point_ {
   coordonnee x;
   coordonnee y;
}Point;

typedef struct Vecteur_ {
   coordonnee x;
   coordonnee y;
}Vecteur;

/******************/
/* initialisation */
/******************/

/* cree le point de coordonnées (x, y) et utilisable pour depuis l'origine vers un point */
Point set_point (coordonnee x, coordonnee y);

/* vecteur correspondant au bipoint AB */
Vecteur vect_bipoint(Point A, Point B);

/*-----------------------------------------------------------------------------------------------------------*/

/*********/
/* Somme */
/*********/

/* somme dess points P1 + P2 */
Point add_point (Point P1, Point P2);

/* somme des vecteurs V1 et V2 */
Vecteur add_vect(Vecteur V1, Vecteur V2);

/*-----------------------------------------------------------------------------------------------------------*/

/********/
/* Mult */
/********/

/* produit du point P par un reel (double) */
Point mult_scalaire_point(Point P, double lambda);

/* produit du vecteur V par un reel (double) */
Vecteur mult_scalaire_vect(Vecteur V, double lambda);

/* Pruduit scalaire entre V1 et V2 */
double prod_scal(Vecteur V1, Vecteur V2);

/*-----------------------------------------------------------------------------------------------------------*/
/********************/
/* Norme et distance */
/********************/

/* renvoie la norme du vecteur V */
double norme(Vecteur V);

/* renvoie la distance entre les point P1 et P2 */
double distance(Point P1, Point P2);
#endif
```

Voici le contenu de geom2d.c :

```
#include <stdio.h>
#include <math.h>
#include "geom2d.h"
#include"types_macros.h"

/******************/
/* initialisation */
/******************/
/* cree le point de coordonnées (x, y) et utilisable pour depuis l'origine vers un point */
Point set_point (coordonnee x, coordonnee y){
    Point P = {x, y};
    return P;
}

/* vecteur correspondant au bipoint AB */
Vecteur vect_bipoint(Point A, Point B){
    Vecteur V = {B.x - A.x, B.y - A.y};
    return V;
}

/*********/
/* Somme */
/*********/

/* somme dess points P1 + P2 */
Point add_point (Point P1, Point P2){
    Point P = {P1.x + P2.x, P1.y + P2.y};
    return P;
}

/* somme des vecteurs V1 et V2 */
Vecteur add_vect(Vecteur V1, Vecteur V2){
    Vecteur V = {V1.x + V2.x, V1.y + V2.y};
    return V;
}

/********/
/* Mult */
/********/

/* produit du point P par un reel (double) */
Point mult_scalaire_point(Point P, double lambda){
    Point Pt = {lambda * P.x, lambda * P.y};
    return Pt;
}

/* produit du vecteur V par un reel (double) */
Vecteur mult_scalaire_vect(Vecteur V, double lambda){
    Vecteur Vect = {lambda * V.x, lambda * V.y};
    return Vect;
}

/* Pruduit scalaire entre V1 et V2 */
double prod_scal(Vecteur V1, Vecteur V2){
    return ((V1.x * V2.x) + (V1.y * V2.y));
}

/*********************/
/* Norme et distance */
/*********************/

/* renvoie la norme du vecteur V */
double norme(Vecteur V){
    return sqrt(prod_scal(V, V));
}

/* renvoie la distance entre les point P1 et P2 */
double distance(Point P1, Point P2){
    return sqrt(pow(P2.x - P1.x, 2.) + pow(P2.y - P1.y, 2.));
}
```

Voici le contenu du fichier test_alea_geom2d.c :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <geom2d.h>




/* renvoie un Double avec 15 chiffre apres la virgule */
double randomDouble()
{
    int max = 100;
    // x est un nombre pseudo aléatoire entre 0 et max
    double x = ((double)rand() / (double)RAND_MAX) * max;
    return x;
}

/* renvoie un double sous forme d'entier par ex : 2. = 2 */
double randomInt()
{
    return (double)(rand() % 100);
}

/* fonction qui verifie chaque fonction une fois grace a 4 coordonnées */
int test_all_fonction(int nb_test, int coef, bool test_double){
    double result, lambda;
    Point A, B, C;
    Vecteur U, W, V;
    coordonnee x1, x2, y1, y2;

    for (int i = 0; i < nb_test; i++) {

        //generation aleatoire de coordonnées
        if (test_double) /* test avec des double */
        {
            x1 = coef * randomDouble();
            x2 = coef * randomDouble();
            y1 = coef * randomDouble();
            y2 = coef * randomDouble();
            lambda = coef * randomDouble();
        }
        else /* test avec des entier positif */
        {
            x1 = coef * randomInt();
            x2 = coef * randomInt();
            y1 = coef * randomInt();
            y2 = coef * randomInt();
            lambda = coef * randomInt();
        }

        /****************************/
        /* tests des fonctions points */
        /****************************/

        A = set_point (x1, y1);
        B = set_point (x2, y2);

        if (A.x != x1 || A.y != y1){
            printf(RED"ERREUR DANS set_point avec le point A = (%f, %f)\n"RESET, x1, y1);
            return 1;
        }

        if (B.x != x2 || B.y != y2){
            printf(RED"ERREUR DANS set_point avec le point B = (%f, %f)\n"RESET, x2, y2);
            return 1;
        }


        C = add_point(A, B);
```

```
        if (C.x != x1 + x2 || C.y != y1 + y2){
            printf(RED"ERREUR DANS add_point avec les points A = (%f, %f) et B = (%f, %f)\n", x1, y1, x2, y2);
            return 1;
        }

        C = mult_scalaire_point(A, lambda);

        if (C.x != lambda * x1 || C.y != lambda * y1){
            printf(RED"ERREUR DANS mult_scalaire_point avec le points A = (%f, %f)\n"RESET, x1, y1);
            return 1;
        }

        result = distance(A, B);

        if (result != sqrt(pow(x1 - x2, 2.) + pow(y1 - y2, 2.))){
            printf(RED"ERREUR DANS norme avec les points A = (%f, %f) et B = (%f, %f)\n"RESET, x1, y1, x2, y2);
            return 1;
        }

        /******************************/
        /* tests des fonctions vecteur */
        /******************************/

        //creation d'un vecteur bipoint
        U = vect_bipoint(A, B);
        if (U.x != x2 - x1 || U.y != y2 - y1){
            printf(RED"ERREUR DANS vect_bipoint avec les points A = (%f, %f) et B = (%f, %f)\n"RESET, x1, y1, x2, y2);
            return 1;
        }

        x1 = U.x; y1 = U.y;

        //creation d'un vecteur uni point
        V.x = A.x;
        V.y = A.y;

        x2 = V.x; y2 = V.y;

        W = add_vect(U, V);
        if (W.x != x1 + x2 || W.y != y1 + y2){
            printf(RED"ERREUR DANS add_vect avec les vecteurs U = (%f, %f) et V = (%f, %f)\n"RESET, x1, y1, x2, y2);
            return 1;
        }

        W = mult_scalaire_vect(U, lambda);
        if (W.x != lambda * x1 || W.y != lambda * y1){
            printf(RED"ERREUR DANS mult_scalaire_vect avec le vecteur U = (%f, %f)\n"RESET, x1, y1);
            return 1;
        }

        result = prod_scal(U, V);
        if (result != x1*x2 + y1*y2){
            printf(RED"ERREUR DANS prod_scal avec les vecteurs U = (%f, %f) et V = (%f, %f)\n"RESET, x1, y1, x2, y2);
            return 1;
        }

        result = norme(U);
        if (result != sqrt(x1*x1 + y1*y1)){
            printf(RED"ERREUR DANS norme avec les vecteurs U = (%f, %f)\n"RESET, x1, y1);
            return 1;
        }

    }

    return 0;
}


int main(void){

    srand(time(NULL));
```

```
    double result;
    int nb_test;

    printf(GRN"Combien de tests voulez-vous effectuer ?\n"RESET);
    scanf("%d", &nb_test);

    result = test_all_fonction(nb_test, 1, true);
    if (result == 0)
        printf(GRN"Tous les tests des réel positif sont bon.\n"RESET);
    else
        return 1;



    /* test avec des double negatif */
    result = test_all_fonction(nb_test, -1, true);
    if (result == 0)
        printf(GRN"Tous les tests des réel négatif sont bon.\n"RESET);
    else
        return 1;



    /* test avec des entier positif */
    result = test_all_fonction(nb_test, 1, false);
    if (result == 0)
        printf(GRN"Tous les tests des entiers positifs sont bon.\n"RESET);
    else
        return 1;


    /* test avec des entiers négatifs */
    result = test_all_fonction(nb_test, -1, false);
    if (result == 0)
        printf(GRN"Tous les tests des entiers négatifs sont bon.\n"RESET);
    else
        return 1;

    //si tout c'est bien passé
    return 0;
}
```

Voici le contenu du fichier test_geom2d.c :

```
#include <stdio.h>
#include <assert.h>
#include <geom2d.h>

//Fichier de tests unitaire du module geom2d

/* test unitaires pour les points */
int unit_test_point(Point P, coordonnee x, coordonnee y)
{
    assert(P.x == x);
    assert(P.y == y);

    return 0;
}


/* test unitaires pour les vecteurs */
int unit_test_vecteur(Vecteur P, coordonnee x, coordonnee y)
{
    assert(P.x == x);
    assert(P.y == y);

    return 0;
}

/* test unitaires pour les double */
int unit_test_double(double d, double res_att)
{
    assert(d == res_att);

    return 0;
}

int main(void)
{
    /* Initialisation des points et vecteurs pour les tests */
    Point P = set_point(0,0); // point origine
    Point P2 = set_point(4, 3); //point P2 = (4,3)
    Vecteur V1 = vect_bipoint(P, P); //vecteur nul
    Vecteur V2 = vect_bipoint(set_point(1, 2), set_point(5, 5)); //vecteur (4,3)

    /* Test sur les points */
    unit_test_point(P, 0, 0);
    unit_test_point(P2,4, 3);
    unit_test_point(add_point(P, P2),P2.x,P2.y);
    unit_test_point(mult_scalaire_point(P, 3), 0, 0);
    unit_test_point(mult_scalaire_point(P2, 5), 5*P2.x, 5*P2.y);
    unit_test_double(distance(P, P2), 5);
    printf(GRN"Tous les tests manuel des point se sont bien passé\n"RESET);

    /* Test sur les vecteurs */
    unit_test_vecteur(V2, 4, 3);
    unit_test_vecteur(V1, 0, 0);
    unit_test_vecteur(add_vect(V1, V2), V2.x, V2.y);
    unit_test_vecteur(mult_scalaire_vect(V1, 4), 0, 0);
    unit_test_vecteur(mult_scalaire_vect(V2, 4),V2.x*4, V2.y*4);
    unit_test_vecteur(mult_scalaire_vect(V2, 3.5), 4*3.5, 3*3.5);
    unit_test_double(prod_scal(V1, V2), 0);
    unit_test_double(prod_scal(V2, V2), 25);
    unit_test_double(norme(V1), 0);
    unit_test_double(norme(V2), 5);
    printf(GRN"Tous les tests manuel des vecteurs se sont bien passé\n"RESET);

    /* test manuel (celui de l'énoncé) */
    Point A, B, C; Vecteur U;
```

```
    A = set_point(1.0, -3.0);
    B = set_point(4.0, 1.0);
    C = add_point (A, B); /* ––> C = ( 5 .0 , –2 .0) */
    if (C.x != 5. || C.y != -2.){
        printf(GRN"ERREUR DANS add_point avec les points A = (1., -3.) et B = (4., 1.)\n"RESET);
        return 1;
    }

    U = vect_bipoint(C, A); /* ––> U = ( –4 .0 , –1 .0) */

    if (U.x != -4. || U.y != -1.){
        printf(GRN"ERREUR DANS vect_bipoint avec les points C = (5., -2.) et A = (1., -3.)\n"RESET);
        return 1;
    }

    printf(GRN"Le test de l'énoncé s'est bien dérouler \n"RESET);


    return 0;
}
```

Résulat de l’exécution :

```
sagsag@MacSag[~/Desktop/SAGAR_MITTAL/SAGAR_MITTAL_2023_2024/S2/MAP401/projet/TACHE2]$ ./TEST/test_geom2d
Tous les tests manuel des point se sont bien passé
Tous les tests manuel des vecteurs se sont bien passé
Le test de l'énoncé s'est bien dérouler
```

```
sagsag@MacSag[~/Desktop/SAGAR_MITTAL/SAGAR_MITTAL_2023_2024/S2/MAP401/projet/TACHE2]$ ./TEST/test_alea_geom2d
Combien de tests voulez-vous effectuer ?
10000
Tous les tests des réel positif sont bon.
Tous les tests des réel négatif sont bon.
Tous les tests des entiers positifs sont bon.
Tous les tests des entiers négatifs sont bon.
```